Alberto Andrés Valdés González.
Degree: Mathematical Engineer.
Work position: ML-Engineer.
Mail: anvaldes@uc.cl/alberto.valdes.gonzalez.96@gmail.com
Location: Santiago, Chile.

Isolation Forest, Kubernetes, Qdrant, Snowflake, Kafka

Setting: Five pieces that often appear in the same ML system: a **model** to detect anomalies (Isolation Forest), a **data warehouse** (Snowflake), an **event stream** (Kafka), a **vector database** (Qdrant) and a **platform** to run everything (Kubernetes).

1. Isolation Forest (anomaly detection)

Idea: anomalies are *few and different*, so they are easier to **isolate**. Build trees by choosing a random feature and a random split value between its min and max, recursively, until each point is alone. An anomaly gets isolated after few splits (short path); a normal point, inside a dense region, needs many.

Score: with $h(x)$ the path length of x in a tree, averaged over the forest as $E[h(x)]$, and n the sub-sample size per tree:

$$s(x,n) = 2^{-\frac{E[h(x)]}{c(n)}}, \qquad c(n) = 2H(n-1) - \frac{2(n-1)}{n}, \qquad H(i) \approx \ln(i) + 0.5772$$

$c(n)$ is the average path length of an unsuccessful search in a binary tree, used to normalize. $s \to 1$: anomaly; $s \approx 0.5$ or less: normal.

Properties: unsupervised (no labels), linear time $O(t \cdot n \log n)$, works well in high dimensions, and needs only two main hyperparameters: number of trees t (default 100) and sub-sample size n (default 256). The `contamination` parameter only sets the decision threshold (expected fraction of anomalies). **Uses:** fraud, intrusion detection, sensor faults, data-quality checks before training.
In `scikit-learn`: `IsolationForest(contamination=0.01).fit(X)`; `predict` returns -1 for anomalies.

2. Kubernetes (container orchestration)

Idea: you declare the **desired state** ("3 replicas of this container image, with 2 CPUs and 4 GB each"), and Kubernetes continuously reconciles the real state of a cluster to match it: it schedules containers on machines, restarts them if they crash, and replaces them if a machine dies.

Main objects: Node (a machine; the **control plane** with API server, scheduler and etcd manages the workers), **Pod** (smallest unit, one or more containers sharing network and storage), **Deployment** (keeps N identical pods running, rolling updates and rollbacks), **Service** (stable DNS name that load-balances across pods; **Ingress** exposes it externally), **ConfigMap / Secret** (configuration and credentials), **Job / CronJob** (run-to-completion tasks, e.g. nightly training).

Autoscaling (HPA): the Horizontal Pod Autoscaler changes the number of replicas to keep a metric at its target:

$$\text{desired replicas} = \left\lceil \text{current replicas} \cdot \frac{\text{current metric}}{\text{target metric}} \right\rceil, \qquad \text{e.g.} \quad \left\lceil 4 \cdot \frac{90\%}{60\%} \right\rceil = 6$$

In ML: serving models (KServe, Seldon), training pipelines (Kubeflow), GPU scheduling. Managed versions: EKS (AWS), GKE (Google), AKS (Azure).

3. Qdrant (vector database)

Idea: store **embeddings** (vectors produced by a model from text, images, etc.) and find the most similar ones to a query vector: the retrieval layer of RAG, semantic search and recommendations.

Data model: a **collection** holds **points**; each point has an `id`, a **vector** (e.g. 768 dimensions) and a **payload** (JSON metadata such as {`"lang": "es", "date": "2026-11-02"`}).

Similarity: cosine, dot product, Euclidean or Manhattan distance. Cosine similarity:

$$\cos(q, x) = \frac{q \cdot x}{\|q\| \cdot \|x\|} \in [-1, 1]$$

Search: brute force costs $O(N \cdot d)$. Qdrant uses **HNSW**, a multi-layer graph linking each vector to its close neighbors; the search greedily descends from a sparse top layer, reaching about $O(\log N)$. It is **approximate** nearest neighbor search: much faster for a small loss in recall.

Features: written in Rust; **payload filtering** during the search (top-5 similar *where* `lang = "es"`); quantization to reduce memory; hybrid dense + sparse search; sharding and replication. Alternatives: Pinecone, Weaviate, Milvus, pgvector.

4. Snowflake (cloud data warehouse)

Idea: a SQL data warehouse (OLAP) as a service, whose key design is the **separation of storage and compute**: **Storage:** data stored once, compressed and columnar, in cloud object storage, split into **micro-partitions** (50–500 MB) with min / max metadata per column to prune what each query reads. **Compute: virtual warehouses** sized XS, S, M, L… (each size doubles resources and cost); many warehouses read the same data concurrently without competing (ETL, BI, data science) and auto-suspend when idle. **Cloud services:** metadata, optimizer, security, transactions.

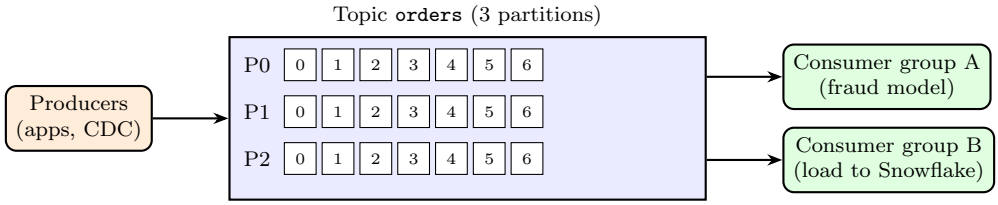
Cost is paid in credits, per second of warehouse running time:

$$\text{cost} \approx \text{credits/hour(size)} \cdot \text{hours running}, \qquad \text{XS} = 1,\ \text{S} = 2,\ \text{M} = 4,\ \text{L} = 8 \text{ credits/hour}$$

Features: Time Travel (query or restore data as of a past moment), **zero-copy cloning** (clone a whole database instantly, only metadata is copied), secure data sharing between accounts, semi-structured data (`VARIANT` for JSON), Snowpark (Python / ML inside Snowflake), Iceberg tables. Alternatives: BigQuery, Redshift, Databricks SQL.

5. Kafka (event streaming)

Idea: a distributed, durable **log** of events. Producers append events, consumers read them at their own pace; new consumers can be added without touching the producer.



Concepts:

- **Topic:** a named stream of events (e.g. `orders`). **Partition:** a topic is split into ordered, append-only logs; partitions are the unit of parallelism. An event with key k goes to partition hash(k) mod $N_{partitions}$, so all events of the same key (e.g. same `customer_id`) keep their order. Order is guaranteed only *within* a partition.
- **Offset:** position of an event in a partition. Consumers commit it to resume after a crash, or move it back to *replay* history.
- **Consumer group:** consumers sharing the work; each partition is read by exactly one consumer of the group, so parallelism $\leq N_{partitions}$.
- **Brokers / replication:** each partition is replicated across servers (e.g. factor 3), so losing one loses no data. **Retention:** events are kept for a time (e.g. 7 days), not deleted when read, unlike a classic queue.

Ecosystem: Kafka Connect, Debezium (CDC), Kafka Streams / Flink, Schema Registry. Managed: Confluent Cloud, Amazon MSK. Alternatives: Kinesis, Pub/Sub, Redpanda.

Comparison:

Tool	Category	Core idea	Typical ML use
Isolation Forest	Unsupervised model	Anomalies need fewer random splits to be isolated	Fraud, monitoring, data-quality filters
Kubernetes	Container orchestration	Declare desired state; the cluster reconciles it	Serve models, run training jobs, autoscale
Qdrant	Vector database	HNSW graph for fast approximate similarity + payload filters	RAG retrieval, semantic search, recommendations
Snowflake	Cloud data warehouse	Separate storage and compute; pay per second of compute	Feature engineering in SQL, training datasets, BI
Kafka	Event streaming	Durable, partitioned, replayable log	Real-time features, CDC, streaming inference

One pipeline with all five: transactions flow through **Kafka**; a service on **Kubernetes** scores them with an **Isolation Forest**; events and scores land in **Snowflake** for analysis and retraining; analysts find similar past cases (embeddings) in **Qdrant**.
